1. Consider the code below. You are supposed to update the value of variable ‘a’ without directly modifying it. That is, you have to modify it through one of the other two variables: b and p. Will the following code modify a? If not, correct it. You are not allowed to introduce new variables.

```
#include<stdio.h>
#include<string.h>
#include<stdlib.h>


void main()
{
   int a, b, *p;

   a = 10;
   b = a;

   p = &a;

   b++;


   printf("a = %d\n", a);

 }
```

Solution:

```
 #include<stdio.h>
#include<string.h>
#include<stdlib.h>


void main()
{
   int a, b, *p;

   a = 10;
   b = a;

   p = &a;

   (*p)++;
```

```
    printf("a = %d\n", a);

 }
```

2. Consider a **double** type array (a) of five elements. Let p be a pointer pointing to array a. Using only p (without using a directly), find if a specific element E is present in array a; if yes, increment E in a by 1. Then, print the updated array using p. E.g., if a is {1.5, 2.4, 5.8, 11.35, 4.4} and E is 2.4, then the modified version of a (after the increment) will be {1.5, **3.4**, 5.8, 11.35, 4.4}.

   Solution:

```
void main()
{

double a[] = {1.5, 2.4, 5.8, 11.35, 4.4}, *p;
int i;

p = a;

for(i = 0; i < 5; i++)
{
          if(*(p+i) == 2.4)
         {
                  (*(p+i))++;
         }
}

printf("The modified array:\n\n");

for(i = 0; i < 5; i++)
{
          printf("%f\n", *(p+i));
}

}
```

3. Spot errors in the program that is supposed to print a modified string in the main through change() and correct it:

```
#include<stdio.h>
#include<string.h>
#include<stdlib.h>


char* change(char *s)//changes the first character of the argument string
{
   char s_out[10];

   s_out = (char*)malloc(sizeof(char)*(strlen(s)+1));

   strcpy(s_out, s);

   s_out[0]--;

   return s_out;

}



void main()
{
   char s[] = "hello", *s_out;

   s_out = change(s);

   printf("%s %s\n", s, s_out);



}
```

Solution:

You can’t dynamically allocate in a static array.

Correct:

```
#include<stdio.h>
#include<string.h>
#include<stdlib.h>


char* change(char *s)//changes the first character of the argument string
{
   char *s_out;

   s_out = (char*)malloc(sizeof(char)*(strlen(s)+1));

   strcpy(s_out, s);

   s_out[0]--;

   return s_out;

}



void main()
{
   char s[] = "hello", *s_out;

   s_out = change(s);

   printf("%s %s\n", s, s_out);

   //free(s_out); //run the code without this line using valgrind --leak-check=full ./string_ptr to see there is mem leak

}
```

4. Spot the issues in the program that is supposed to print an incremented version of a variable:

```
#include<stdio.h>
#include<string.h>
#include<stdlib.h>
```

```
void main()
{
   int *a;

   *a = 10;

   *a++;

   printf("%d\n", *a);
```

Solution:

You should first store the value in a variable and then dereference it through a pointer. In addition, * has a higher preference than ++.

Correct:

```
#include<stdio.h>
#include<string.h>
#include<stdlib.h>




void main()
{
   int *a, v = 10;

   a = &v;//recommended

   (*a)++;

   printf("%d\n", *a);



}
```

5. Consider a database of student records (stored in a structure): name (a multi-word string), id (integer), marks in three subjects as a static array of floating point numbers, average marks (float), and grade letter (string).
   Take name, id, and marks in three subjects as input from the user in a function inputStudentData() that takes a pointer to an array of structures as an argument.
   Then pass this pointer as an argument to the function calculateStatistics() to compute the average marks and grade letter (for the latter use IISER-K standard grade boundaries) in the structure.
   Display all information for all the students by calling a function printallStudents() that takes the same pointer as an argument.

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

#define MAX_STUDENTS 5
#define MAX_NAME_LENGTH 30
#define MAX_GRADE_LENGTH 3  // For "A+" or "B+"

// Structure definition with grade_letter as string
typedef struct {
    char name[MAX_NAME_LENGTH];
    int id;
    float grades[3];  // Three subjects: CS2201, CS3101, CS3201
    float average;
    char grade_letter[MAX_GRADE_LENGTH];  // Now a string: "D", "C", "B", "B+", "A",
"A+"
} Student;

// Function prototypes using pointer notation
void inputStudentData(Student *student);
void calculateStatistics(Student *student);
void printStudentInfo(const Student *student);
void printAllStudents(Student *students, int count);

int main() {
    Student students[MAX_STUDENTS];
    Student *ptr = students;  // Pointer to array of structures


    // Input data for each student using pointer arithmetic
    for (int i = 0; i < MAX_STUDENTS; i++) {
        printf("Enter details for Student %d:\n", i + 1);
        inputStudentData(ptr + i);  // Pointer arithmetic
```

```
        printf("\n");
    }

    // Calculate statistics for each student
    for (int i = 0; i < MAX_STUDENTS; i++) {
        calculateStatistics(ptr + i);
    }

    // Display all students
    printf("\n=== Student Report ===\n");
    printAllStudents(ptr, MAX_STUDENTS);


    return 0;
}

// Function to input student data using pointer notation
void inputStudentData(Student *student) {
    printf("Enter name: ");
    fgets(student->name, MAX_NAME_LENGTH, stdin);
    student->name[strcspn(student->name, "\n")] = '\0';  // Remove newline

    printf("Enter ID: ");
    scanf("%d", &student->id);

    printf("Enter grades (CS2201, CS3101, CS3201): ");
    for (int i = 0; i < 3; i++) {
        scanf("%f", &(student->grades[i]));  // Using pointer notation
    }
    while(getchar() != '\n');  // Clear input buffer
}

// Function to calculate statistics using pointer notation
void calculateStatistics(Student *student) {
    float sum = 0;

    // Calculate average
    for (int i = 0; i < 3; i++) {
        sum += student->grades[i];
    }
    student->average = sum / 3;

    // Determine grade letter as string
    if (student->average >= 90)
```

```
        strcpy(student->grade_letter, "A+");
    else if (student->average >= 80)
        strcpy(student->grade_letter, "A");
    else if (student->average >= 70)
        strcpy(student->grade_letter, "B+");
    else if (student->average >= 60)
        strcpy(student->grade_letter, "B");
    else if (student->average >= 50)
        strcpy(student->grade_letter, "C");
    else if (student->average >= 40)
        strcpy(student->grade_letter, "D");
    else
        strcpy(student->grade_letter, "F");
}

// Function to print student information using pointer notation
void printStudentInfo(Student const *student) {
    printf("Name: %s\n", student->name);
    printf("ID: %d\n", student->id);
    printf("Grades: %.1f, %.1f, %.1f\n",
           student->grades[0], student->grades[1], student->grades[2]);
    printf("Average: %.2f\n", student->average);
    printf("Grade: %s\n\n", student->grade_letter);  // %s instead of %c
}

// Function to print all students using pointer notation
void printAllStudents(Student *students, int count) {
    for (int i = 0; i < count; i++) {
        printf("Student %d:\n", i + 1);
        printStudentInfo(students + i);
    }
 }
```

6. Write a program to dynamically allocate an array of strings, where you do not know the number of strings (and each string or the length) in advance.

   Solution:

```
#include<stdio.h>
#include<string.h>
#include<stdlib.h>
```

```
void main()
{
        char **str, name[50];
        int no, j;

        printf("How many names?\n");

        scanf("%d", &no);

        str = (char**)malloc(no*sizeof(char*));

        for(j = 0; j < no; j++)
        {
                printf("Name - %d?  ", j+1);
                scanf("%s", name);
                str[j] = (char*)malloc(sizeof(char)*(strlen(name)+1));
                strcpy(str[j], name);

        }

        for(j = 0; j < no; j++)
        {
                printf("name - %d is %s\n", j, str[j]);
        }

}
```

7. Consider a database of students allocated dynamically. Each student has an id and some courses, stored as a structure element. Here, both id (as a pointer to char) and courses (as a pointer-to-pointer to char) are allocated dynamically, where the courses (and the number of such courses) can be different for different students. Also, you don't know in advance the number of students, which you need to ask the user. Write a program that takes the aforementioned information from the user for some students and prints them all via a display function print_stu() that takes the database (as a pointer to a structure) as arguments.

Solution:

```
#include<stdio.h>
#include<stdlib.h>
#include<string.h>
```

```
typedef struct
{
	char *stuid;
	int nc;
	char **courseids;

}STUDENT;


void print_stu(STUDENT *s, int num_stu)
{
	int i, j;

	printf("The students:\n\n");

	for(i = 0; i < num_stu; i++)
	{
		printf("Student - %d\n", i+1);
		printf("%s\n", s[i].stuid);

		for(j = 0; j < s[i].nc; j++)
		{
			printf("%s \n", s[i].courseids[j]);



		}

		printf("\n\n");


	}

}

void main()
{
	STUDENT *studs;
	int n, i, j, nc;
	char sid[10], cid[10];

	printf("No of students?\n");
```

```
        scanf("%d", &n);

        studs = (STUDENT*)malloc(sizeof(STUDENT)*n);

        for(i = 0; i < n; i++)
        {
                printf("Student - %d:\n \n", i+1);

                printf("Id?\n");

                scanf("%s", sid);

                studs[i].stuid = (char*)malloc(sizeof(char)*(strlen(sid)+1));
                strcpy(studs[i].stuid, sid);

                printf("No of courses?\n");

                scanf("%d", &studs[i].nc);

                studs[i].courseids = (char**)malloc(sizeof(char*)*studs[i].nc);

                for(j = 0; j < studs[i].nc; j++)
                {
                        printf("Course - %d\n", j+1);
                        scanf("%s", cid);
                        studs[i].courseids[j] = (char*)malloc(sizeof(char*)*(strlen(cid)+1));
                        strcpy(studs[i].courseids[j], cid);


                }

        }


        print_stu(studs, n);

}
```

8. The following code is supposed to print all the elements in the function in f(). However, the code is wrong, and please correct it. Note that the number of elements in the array has to be computed automatically in main().

   #include<stdio.h>

```
void f(int *arr)
{
	int i, n = sizeof(arr);

	printf("n = %d\n", n);

	for(i = 0; i < n; i++)
	{
		printf("ele - %d\n", *(arr+i));
	}

}

void main()
{

	int array[] = {1, 2, 3, 4, 5};

	f(array);

}
```

Solution:

```
#include<stdio.h>

void f(int *arr, int n)// The number of elements has to be passed as an argument; however, this is not necessary for a string, where the last element is always '\0'
{
	int i;

	printf("n = %d\n", n);

	for(i = 0; i < n; i++)
	{
		printf("ele - %d\n", *(arr+i));
	}

}

void main()
{
```

```
        int array[] = {1, 2, 3, 4, 5};

        f(array, sizeof(array)/sizeof(int));

}
```

9. Use Valgrind to check for memory leaks in the following code:

```
#include<stdio.h>
#include<string.h>
#include<stdlib.h>


void main()
{
   int *a;

   a = (int*)malloc(5*sizeof(int));

   free(a);


   printf("%d\n", a[0]);




}
```

Solution:

valgrind --leak-check=full ./prog

10. Consider an acid test situation where a stranger is asked to solve some tasks to check whether the person is an agent of a secret detective agency named KHOPAT. The stranger is asked to solve three tasks named *epanq, opanq,* and *jhopanq* – one after the other. If any of the tasks are performed wrongly, the person is declared “Trespasser” immediately (without the need to perform the remaining tasks, if any), followed by

serious consequences, while if all the tasks are performed correctly, the person is warmly welcomed.

To make the final decision (Trespasser or not), design a C function named *ghochanq*() that takes pointers to functions epanq, opanq, and jhopanq as arguments. Now, given two randomly generated integers x and y, epanq, opanq, and jhopanq functions are defined as below:

n1 = epanq(x, y) = 2x + y, n2 = opanq(x, y) = 3y+x and n3 = jhopanq(n1, n2) is a two-digit integer composed of the last two digits of the integers n1 and n2 in this order.

E.g if x = 5, y = 6, n1 = 16, n2 = 23 and n3 = 63.

Furthermore, you have to generate x and y randomly within any range of your choice. In C, the following piece of code generates a random integer within a range [min, max]:

```
#include<stdlib.h>
#include <time.h>

…

void main()
{
        int max = 10, min = 5, rd_num;


  srand(time(0)); //used to initialize or set the seed for the 'rand()' function, allowing us to generate different sequences of random numbers.

        rd_num = rand() % (max - min + 1) + min;// rd_num is the random number between min and max

}
```

Solution:

```
#include<stdio.h>
#include <stdlib.h>
#include <time.h>


int epanq(int a, int b)
{
        return 2*a + b;
```

```
}

int opanq(int a, int d)
{
	return 3*d + a;
}

int jhopanq(int x, int y)
{
	return 10*(x%10) + y%10;
}

int ghochanq(int (*f1)(), int (*f2)(), int (*f3)(), int a, int b)
{
	int s1, s2, s3;

	printf("Given the numbers %d and %d perform the epanq operation and input the
results:\n\n", a, b);

	scanf("%d", &s1);

	printf("s1 calculated = %d\n", f1(a, b));

	if(s1 != f1(a, b))
	{
		return 0;
	}



	printf("Given the numbers %d and %d perform the opanq operation and input the
results:\n\n", a, b);

	scanf("%d", &s2);

	printf("s2 calculated = %d\n", f2(a, b));

	if(s2 != f2(a, b))
	{
		return 0;
	}


	printf("Finally perform the jhopanq operation and input the results:\n\n");
```

```
        scanf("%d", &s3);

        printf("s3 calculated = %d\n", f3(s1, s2));

        if(s3 != f3(s1, s2))
        {
                return 0;
        }

        return 1;

}

void main()
{
  int max = 10, min = 5, rd_num, rd_num1, x, y, final_out;

  int (*f1)(), (*f2)(), (*f3)();

  srand(time(0));

  rd_num = rand() % (max - min + 1) + min;

  printf("%d \n", rd_num);

  //seed = time(0);

  rd_num1 = rand() % (max - min + 1) + min;

  printf("%d \n", rd_num1);

  f1 = epanq;
  f2 = opanq;
  f3 = jhopanq;

  //x = f1(rd_num, rd_num1);
  //y = f2(rd_num, rd_num1);

  //final_out = f3(x, y);

  //printf("%d %d %d\n", x, y, final_out);

  if(ghochanq(f1, f2, f3, rd_num, rd_num1) == 0)
```

```
 {
        printf("Trespasser alert!\n\n");
 }
 else
 {
        printf("Welcome!!\n");
 }


}
```

Linked lists

1. Write a program in C to make two linked lists from a linked list L: one having the numbers of L divisible by N and the other containing the rest.

```
void splitN(struct node *start1,struct node **start2,struct node **start3, int N)
{
        struct node *p1=start1, *p2=*start2, *p3=*start3;

        while(p1!=NULL)
        {
                if( (p1->info)%N != 0 )
                {
                        if(*start2==NULL)
                                *start2 = addatbeg(*start2,p1->info);
                        else
                                *start2 = addatend(*start2,p1->info);
                }
                else
                {
                        if(*start3==NULL)
                                *start3 = addatbeg(*start3,p1->info);
                        else
                                *start3 = addatend(*start3,p1->info);
                }
                p1=p1->link;
        }
}
```

2. Write a program in C to check whether two singly linked lists are identical.
3. Write a program in C to delete duplicates from a sorted single-linked list.

4. Consider that two sets are stored in two singly linked lists: L1: {1, 2, 3, 4, 5} and L2: {1, 5, 8, 9} (as examples). Store the intersection and union of these two sets in two new linked lists: L_intersect and L_union, respectively. Since the contents are sets, the order of the elements in the linked lists is not important.
5. A polynomial (e.g., $2x^3 + 5x^2 + 44$) can be expressed as a singly linked list where each term (e.g., $2x^3$) is a node that stores the coefficient (2) and the exponent (3) along with the pointer. With this notion, implement the addition and multiplication of two such polynomials, each expressed as a singly linked list.

6. Write a program in C to remove a node in a doubly linked list if its previous and next nodes have the same info.

```
struct node* delIfPrevNextSameInfo(struct node *start)
{
        struct node *tmp;

        tmp=start->next;

        while(tmp->next!=NULL)
        {
                if(tmp->prev->info == tmp->next->info)
                {
                        tmp->prev->next=tmp->next;
                        tmp->next->prev=tmp->prev;
                        free(tmp);
                        display(start);
                        return start;
                }
                tmp=tmp->next;
        }

}
```

7. Take the name of the user (e.g., Narendra Damodardas Modi) as input and store it character-wise, that is – one character in one node (space is also stored in one node), in a doubly linked list (L), and from this list store the abbreviated name character-wise, e.g., N. D. Modi in another doubly linked (L_abbr). Finally, print the contents of L_abbr.